

2.4 Matrix Multiplication



Core skills you'll master in this section:

- ▶ Matrix multiplication: multiplying two matrices together to produce a new matrix.
- ▶ Shape-matching rule: the left matrix's number of columns must equal the right matrix's number of rows; the product's shape is determined by the left matrix's rows and the right matrix's columns.
- ▶ Performing matrix multiplication in NumPy: can be done using `@` or `np.matmul()`.
- ▶ Elements of the product: each one is computed as the dot product of a corresponding row vector and column vector.
- ▶ Matrix multiplication is generally not commutative.

Matrix multiplication may well be the single most important operation in linear algebra; it represents multiplying two matrices together to produce a new matrix. When doing matrix multiplication, pay special attention to shape matching; also, matrix multiplication is generally not commutative.

Given the importance of matrix multiplication, this volume devotes several sections to introducing it from different angles; in this section, let's take our first look at matrix multiplication.

What Is Matrix Multiplication?

Matrix multiplication is a fundamental operation in linear algebra, used to multiply two matrices together to produce a new matrix.

⚠ Note that sometimes the result of matrix multiplication can be a 1×1 matrix, which we treat as a scalar. You'll see this shortly.

Suppose we have two matrices **A** and **B**. As shown in Figure 1, matrix **A**'s shape is $m \times p$ — that is, **A** has m rows and p columns; matrix **B**'s shape is $p \times n$ — that is, **B** has p rows and n columns.

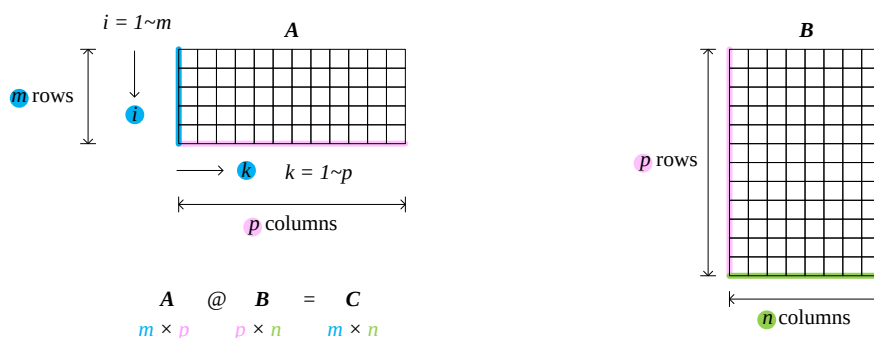
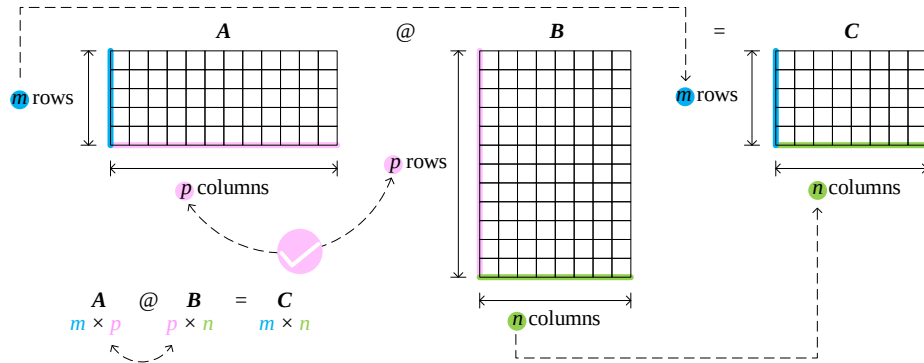


Figure 1. The Shapes of Matrices A and B

Then, as shown in Figure 2, matrix A and B 's product $C = AB = A @ B$ is a matrix of shape $m \times n$.

⚠ Note that this book will also write matrix multiplication AB as $A @ B$, a notation chosen to stay consistent with the syntax of Python scientific computing libraries such as NumPy.

Figure 2. The Matrix Product $A @ B$

From a shape-matching perspective, matrix multiplication $AB = A @ B$ requires the left matrix A 's number of columns (p) to equal the right matrix B 's number of rows (p); otherwise, the matrix multiplication cannot be carried out.

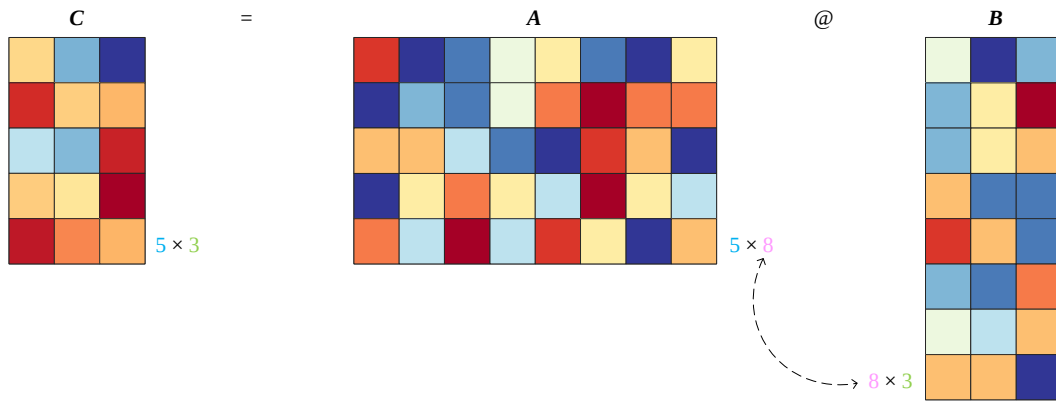
The product of matrices A and B —call it C —inherits A 's number of rows (m) and B 's number of columns (n); meanwhile A 's number of columns (p) and B 's number of rows (p) effectively “cancel out.”

Here's a handy way to remember it: looking at the matrix shapes, $(m \times p) @ (p \times n)$ — the (p) sandwiched in the middle gets “canceled out.”

Computing Matrix Multiplication with NumPy

Let's see how to carry out the matrix multiplication above using NumPy. In NumPy, matrix multiplication can be done with the simple `@` operator, or with the `np.matmul()` or `np.dot()` functions. Code 1 uses the `@` operator to perform matrix multiplication, and visualizes it with a heatmap (as shown in Figure 3).

Looking at the matrix shapes, $(5 \times 8) @ (8 \times 3)$ — the (8) sandwiched in the middle gets “canceled out,” so the result of the matrix multiplication has shape (5×3) .

Figure 3. Visualizing the Matrix Product $A @ B$

Next, let's walk through the key lines in Code 1.

You should already be familiar with **a** — these three lines. After setting the random seed, we use `numpy.random.randint()` to create two matrices.

b Use `@` to perform matrix multiplication; this line is equivalent to `np.matmul(A, B)`.

c This line creates a Matplotlib figure object `fig`, and within it generates an array of axes `axs` containing 1 row and 5 columns of subplots. Here, `figsize=(8, 3)` specifies the figure's size — 8 inches wide and 3 inches tall.

d Activate the first subplot's axis, `axs[0]`, as the current axis, so we can plot on it.

e Use `seaborn.heatmap()` to draw the heatmap.

Here, `C` is the matrix (2D array) to visualize.

`cmap = 'RdYlBu_r'` specifies the colormap.

`xticklabels=[]` and `yticklabels=[]` are used to hide the tick labels on the x-axis and y-axis.

`cbar_kws={"orientation": "horizontal"}` sets the color bar to a horizontal orientation.

f This line ensures the subplot has equal aspect ratio.

g Use `title()` from `matplotlib.pyplot` (abbreviated as `plt`) to add a title to the subplot.

h Hide this subplot's axes so that it displays only the equals sign, without a border or tick marks.

Code 1. Visualizing Matrix Multiplication with a Heatmap |  LA_Ch02_04_01.ipynb

```

## 初始化
import numpy as np
import seaborn as sns
from matplotlib import pyplot as plt

## 创建矩阵A和B
a np.random.seed(88)
A = np.random.randint(0, 10, (5, 8))
B = np.random.randint(0, 10, (8, 3))

## 矩阵乘法 A@B
b C = A @ B

## 可视化矩阵乘法
c fig, axs = plt.subplots(1, 5, figsize=(8, 3))

d plt.sca(axs[0])
e ax = sns.heatmap(C, cmap='RdYlBu_r',
                  xticklabels = [], yticklabels = [],
                  cbar_kws={"orientation": "horizontal"})
f ax.set_aspect("equal")
g plt.title('C')

h plt.sca(axs[1])
plt.title('=') # 绘制等号
plt.axis('off')

plt.sca(axs[2])
ax = sns.heatmap(A, cmap='RdYlBu_r',
                  xticklabels = [], yticklabels = [],
                  cbar_kws={"orientation": "horizontal"})
ax.set_aspect("equal")
plt.title('A')

plt.sca(axs[3])
plt.title('@') # 绘制乘号
plt.axis('off')

plt.sca(axs[4])
ax = sns.heatmap(B, cmap='RdYlBu_r',
                  xticklabels = [], yticklabels = [],
                  cbar_kws={"orientation": "horizontal"})
ax.set_aspect("equal")
plt.title('B')

```



Computing Each Element

Next, let's look at exactly how to compute each element of matrix C .

Matrix C 's element c_{ij} at row i , column j , is computed as

This PDF file is the author's draft, published to make it convenient for readers to study on mobile devices; the final content is subject to the printed edition published by Tsinghua University Press.

Copyright belongs to Tsinghua University Press. Please do not use it commercially, and please credit the source when quoting.
Code and PDF file downloads: <https://github.com/Visualize-ML>

The companion video lectures for this book are all published on Bilibili — 生姜 DrGinger: <https://space.bilibili.com/513194466>
Comments and corrections are welcome. The book's dedicated email address: jiang.visualize.ml@gmail.com

$$c_{i,j} = a_{i,1}b_{1,j} + a_{i,2}b_{2,j} + \cdots + a_{i,k}b_{k,j} + \cdots + a_{i,p}b_{p,j} = \sum_{k=1}^p a_{i,k}b_{k,j}$$

As shown in Figure 4, simply put, c_{ij} is matrix A 's row i , $\mathbf{a}^{(i)}$, dotted with matrix B 's column j , $\mathbf{b}_j$ — that is, the sum of the products of their corresponding elements.

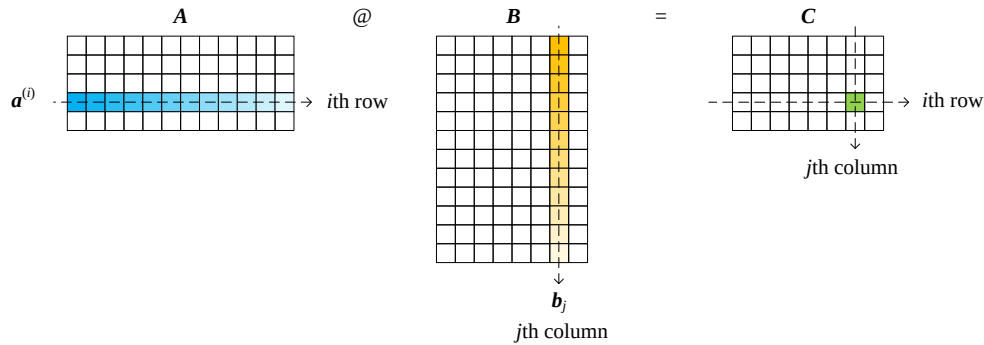


Figure 4. Computing Each Element of the Matrix Product $\mathbf{C} = \mathbf{AB}$

You should notice that $\mathbf{a}^{(i)} @ \mathbf{b}_j$ is also a matrix multiplication, namely

$$c_{i,j} = \mathbf{a}^{(i)} @ \mathbf{b}_j$$

As shown in Figure 5, the $1 \times p$ row vector $\mathbf{a}^{(i)}$ and the $p \times 1$ column vector $\mathbf{b}_j$ multiply to give a 1×1 matrix; and since a 1×1 matrix has only a single element, it's equivalent to a scalar.

⚠ The equation above also tells us that the result of a matrix multiplication can be a scalar (a 1×1 matrix)! This is especially worth noting.

We can also write in the form of a vector inner product

$$c_{i,j} = (\mathbf{a}^{(i)})^T \cdot \mathbf{b}_j$$

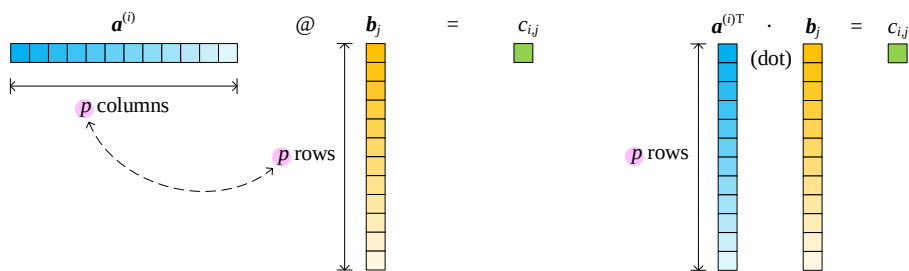


Figure 5. The Matrix Product $\mathbf{a}^{(i)} @ \mathbf{b}_j = c_{ij}$

And the matrix product $\mathbf{C} = \mathbf{AB}$ amounts to performing $m \times n$ row-times-column multiplications — in other words, $m \times n$ vector inner products.

We can understand it this way: matrix multiplication $\mathbf{AB}$ is nothing more than a set of dot products between A 's row vectors and B 's column vectors.

An Example

Let's compute a matrix multiplication with a simple example below.

Given the following matrices A and B

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}, B = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}$$

As shown in Figure 6, matrix A 's shape is 2×3 , matrix B 's shape is 3×2 , so the result matrix $C = AB$ has shape 2×2 .

Looking at the matrix shapes, $(2 \times 3) @ (3 \times 2)$ — the (3) sandwiched in the middle gets “canceled out,” so the result of the matrix multiplication has shape (2×2) .

$$\begin{array}{ccc}
 A & @ & B \\
 \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} & & \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix} \\
 & = & C \\
 & & \begin{bmatrix} 14 & 32 \\ 32 & 77 \end{bmatrix}
 \end{array}$$

Figure 6. The Matrix Product AB , A 's shape is 2×3 , B 's shape is 3×2

Next, let's compute each element of the matrix multiplication $C = AB$

$$\begin{cases}
 c_{1,1} = a_{1,1} \cdot b_{1,1} + a_{1,2} \cdot b_{2,1} + a_{1,3} \cdot b_{3,1} = 1 \times 1 + 2 \times 2 + 3 \times 3 = 1 + 4 + 9 = 14 \\
 c_{1,2} = a_{1,1} \cdot b_{1,2} + a_{1,2} \cdot b_{2,2} + a_{1,3} \cdot b_{3,2} = 1 \times 4 + 2 \times 5 + 3 \times 6 = 4 + 10 + 18 = 32 \\
 c_{2,1} = a_{2,1} \cdot b_{1,1} + a_{2,2} \cdot b_{2,1} + a_{2,3} \cdot b_{3,1} = 4 \times 1 + 5 \times 2 + 6 \times 3 = 4 + 10 + 18 = 32 \\
 c_{2,2} = a_{2,1} \cdot b_{1,2} + a_{2,2} \cdot b_{2,2} + a_{2,3} \cdot b_{3,2} = 4 \times 4 + 5 \times 5 + 6 \times 6 = 16 + 25 + 36 = 77
 \end{cases}$$

Figure 7 shows how to compute each element of matrix C via matrix multiplication.

$$\begin{array}{ccc}
 a^{(1)} \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} @ b_1 \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix} = c_{1,1} \begin{bmatrix} 14 & 32 \\ 32 & 77 \end{bmatrix} & a^{(1)} \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} @ b_2 \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix} = c_{1,2} \begin{bmatrix} 14 & 32 \\ 32 & 77 \end{bmatrix} \\
 a^{(2)} \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} @ b_1 \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix} = c_{2,1} \begin{bmatrix} 14 & 32 \\ 32 & 77 \end{bmatrix} & a^{(2)} \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} @ b_2 \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix} = c_{2,2} \begin{bmatrix} 14 & 32 \\ 32 & 77 \end{bmatrix}
 \end{array}$$

Figure 7. Computing Each Element of the Matrix Product AB , A 's shape is 2×3 , B 's shape is 3×2



LA_02_04_02.ipynb computes each element of the matrix multiplication above — please study it on your own.

Not Commutative

Also, you've probably already noticed that the left/right positions in matrix multiplication matter a great deal and cannot be swapped freely. This is because matrix multiplication is, in general, not commutative.

First, $A @ B$ satisfying the rules for matrix multiplication doesn't mean $B @ A$ also satisfies the rules for matrix multiplication (as shown in Figure 8).

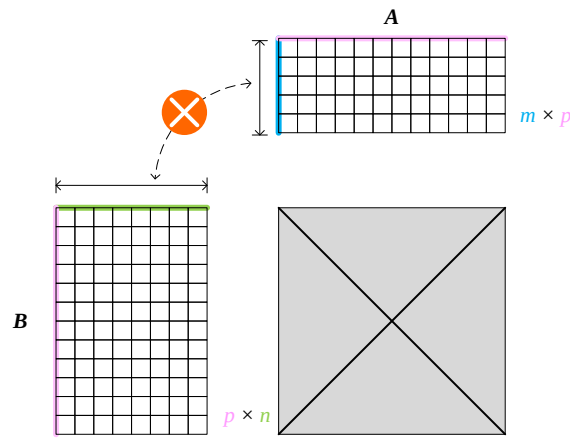


Figure 8. $B @ A$ Does Not Satisfy the Rules for Matrix Multiplication

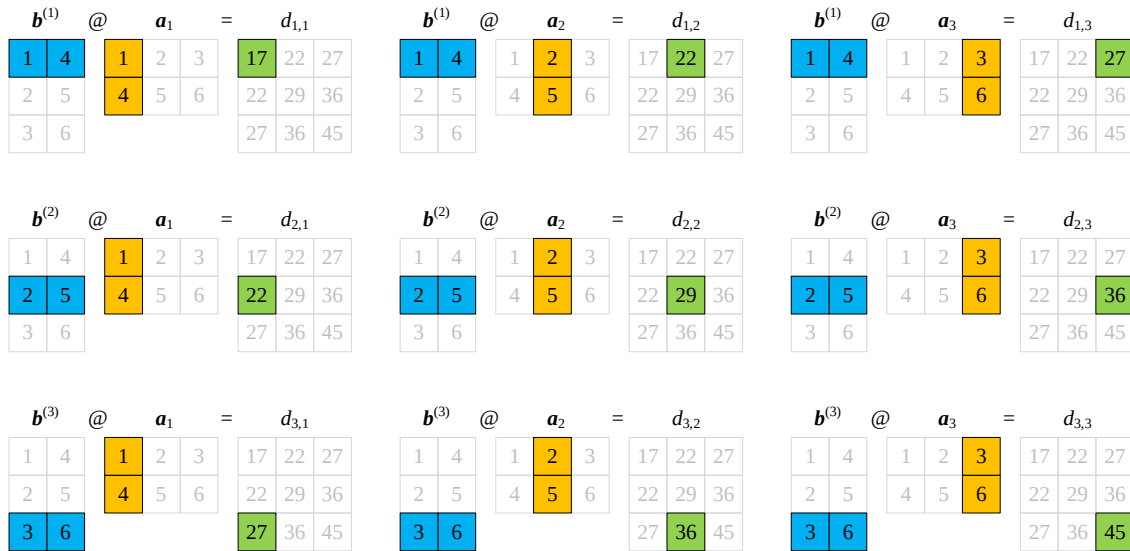
Even if $B @ A$ also satisfies the rules for matrix multiplication, that doesn't mean $A @ B$ and $B @ A$ give the same result.

For example, in the case below, $B @ A$'s result is clearly different from $A @ B$

$$D = B @ A = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix} @ \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} = \begin{bmatrix} 17 & 22 & 27 \\ 22 & 29 & 36 \\ 27 & 36 & 45 \end{bmatrix}$$

Looking at the matrix shapes, $(3 \times 2) @ (2 \times 3)$ — the (2) sandwiched in the middle gets “canceled out,” so the result of the matrix multiplication has shape (3×3) .

Figure 9 shows how to compute each element of $B @ A$.

Figure 9. Computing Each Element of the Matrix Product $B @ A$; A 's shape is 2×3 , B 's shape is 3×2

? Try computing the matrix multiplication above by hand. You may have noticed that $A @ B$ and $B @ A$ are both symmetric matrices — is this a coincidence, or is there a deeper reason?

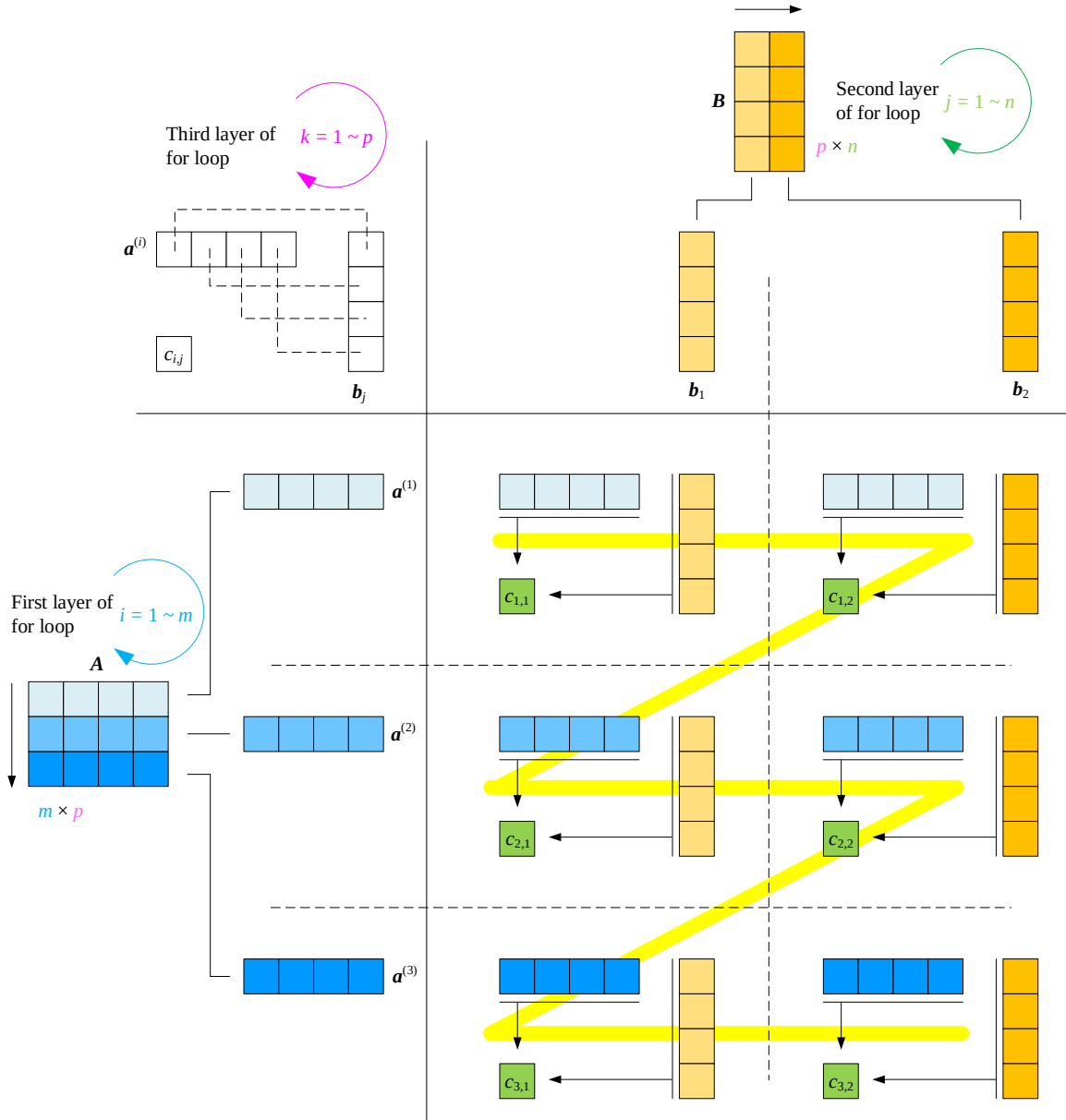
Computing Matrix Multiplication with a Custom Python Function

Code 2 defines a custom Python function to compute matrix multiplication.

- Define a custom function named `matrix_multiplication`, which can be called repeatedly. It takes two input parameters, `A` and `B` — you pass in two matrices, and it returns the result of their matrix multiplication.
 - First, get matrix `A`'s dimensions — `.shape` is a built-in attribute of NumPy arrays that returns a matrix's "number of rows" and "number of columns." Then get matrix `B`'s dimensions. `p_B` denotes `B`'s number of rows, and `n` denotes `B`'s number of columns. Now we know the sizes of both input matrices.
 - This is a conditional check: if matrix `A`'s number of columns `p_A` does not equal matrix `B`'s number of rows `p_B`, the condition for matrix multiplication is not met. If the condition holds — meaning the two matrices' shapes don't match — an error is raised to warn the user that the multiplication cannot be performed. `raise` means to throw an error, and `ValueError` is the error type; this pattern is part of Python's error-handling mechanism, used to keep the program from crashing.
 - Use `numpy.zeros()` to create an all-zero matrix `C`, with shape `m` rows by `n` columns — the same size as the final result matrix.
- Next, we use three nested for loops to compute each element of the matrix multiplication, as shown in Figure 10.
- `i` is the first for loop; `i` is used to iterate over the rows of matrix `A`. `range(m)` produces integers from 0 to `m - 1`, used to visit each row of `A` one by one.
 - `j` is the second for loop; `j` is used to iterate over the columns of matrix `B`. `range(n)` produces integers from 0 to `n - 1`, used to visit each column of `B` one by one.
 - `k` is the third for loop; `k` iterates over `A`'s number of columns, which is the same as `B`'s number of rows. This is the step in matrix multiplication where corresponding elements are multiplied and then accumulated. `C[i, j] += A[i, k] * B[k, j]` takes `A`'s element at row `i`, column `k`, and `B`'s element at row `k`, column `j`, multiplies them, and adds the result to `C[i, j]`.

The relationship among these three nested for loops is shown in Figure 10.

- h defines matrices A and B .
- i calls the custom function to compute AB and BA .

Figure 10. The Matrix Multiplication Rule AB , Implemented with Three Nested for Loops

Code 2. Computing Matrix Multiplication with a Custom Python Function, Implemented with Three Nested for Loops |  LA_02_04_03.ipynb

```

## 初始化
import numpy as np

## 自定义函数
def matrix_multiplication(A, B):

    # 获取矩阵 A 和 B 的形状
    m, p_A = A.shape
    p_B, n = B.shape

    # 检测矩阵形状是否符合矩阵乘法规则
    if p_A != p_B:
        raise ValueError('Dimensions do not match')

    # 初始化结果矩阵 C, 形状 (m, n), 初始值设为 0
    C = np.zeros((m, n))

    # 进行矩阵乘法计算, 使用三层 for 循环
    for i in range(m):      # 遍历 A 的行
        for j in range(n):  # 遍历 B 的列
            for k in range(p_A): # 遍历 A 的列 / B 的行
                C[i, j] += A[i, k] * B[k, j] # 逐元素累加

    return C

## 定义矩阵
A = np.array([[1, 2, 3],
              [4, 5, 6]])
B = A.T

## 矩阵乘法
matrix_multiplication(A, B)
matrix_multiplication(B, A)

```

It's hard to overstate how important matrix multiplication is in linear algebra. This section has only scratched the surface of the world of matrix multiplication; next, we'll focus on the geometric intuition behind matrix multiplication and use a geometric perspective to explain many of its properties. After that, we'll explore matrix multiplication from the perspective of block matrices, gaining yet another way to understand it. All of these perspectives will play a crucial role in the linear algebra topics that follow.



Please use tools such as DeepSeek/ChatGPT to complete the following exercises for this section.

Q1. Determine whether the shapes of each pair of matrices below allow $A @ B$ or $B @ A$ to be computed.

► $A_{2 \times 2} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, B_{2 \times 2} = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$

This PDF file is the author's draft, published to make it convenient for readers to study on mobile devices; the final content is subject to the printed edition published by Tsinghua University Press.

Copyright belongs to Tsinghua University Press. Please do not use it commercially, and please credit the source when quoting.
Code and PDF file downloads: <https://github.com/Visualize-ML>

The companion video lectures for this book are all published on Bilibili — 生姜 DrGinger: <https://space.bilibili.com/513194466>
Comments and corrections are welcome. The book's dedicated email address: jiang.visualize.ml@gmail.com

$$\mathbf{A}_{1 \times 3} = [1 \quad 2 \quad 3], \mathbf{B}_{3 \times 1} = \begin{bmatrix} 4 \\ 5 \\ 6 \end{bmatrix}$$



$$\mathbf{A}_{3 \times 2} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix}, \mathbf{B}_{2 \times 2} = \begin{bmatrix} 7 & 8 \\ 9 & 10 \end{bmatrix}$$



$$\mathbf{A}_{2 \times 3} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}, \mathbf{B}_{3 \times 2} = \begin{bmatrix} 7 & 8 \\ 9 & 10 \\ 11 & 12 \end{bmatrix}$$



$$\mathbf{A}_{2 \times 2} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \mathbf{B}_{2 \times 1} = \begin{bmatrix} 5 \\ 6 \end{bmatrix}$$



Q2. Each pair of matrices below has a shape that supports both $\mathbf{A} @ \mathbf{B}$ and $\mathbf{B} @ \mathbf{A}$. Using a suitable method (by hand or by programming), determine whether $\mathbf{A} @ \mathbf{B}$ and $\mathbf{B} @ \mathbf{A}$ are equal.

$$\mathbf{A}_{2 \times 2} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \mathbf{B}_{2 \times 2} = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$$



$$\mathbf{A}_{1 \times 2} = [1 \quad 2], \mathbf{B}_{2 \times 1} = \begin{bmatrix} 3 \\ 4 \end{bmatrix}$$



$$\mathbf{A}_{2 \times 2} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \mathbf{B}_{2 \times 2} = \begin{bmatrix} 2 & 3 \\ 4 & 5 \end{bmatrix}$$



$$\mathbf{A}_{2 \times 2} = \begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix}, \mathbf{B}_{2 \times 2} = \begin{bmatrix} 3 & 4 \\ 4 & 3 \end{bmatrix}$$



$$\mathbf{A}_{2 \times 2} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}, \mathbf{B}_{2 \times 2} = \begin{bmatrix} 2 & 2 \\ 2 & 2 \end{bmatrix}$$



$$\mathbf{A}_{2 \times 2} = \begin{bmatrix} 1 & 0 \\ 0 & 2 \end{bmatrix}, \mathbf{B}_{2 \times 2} = \begin{bmatrix} 3 & 0 \\ 0 & 4 \end{bmatrix}$$



$$\mathbf{A}_{2 \times 2} = \begin{bmatrix} \cos(30^\circ) & -\sin(30^\circ) \\ \sin(30^\circ) & \cos(30^\circ) \end{bmatrix}, \mathbf{B}_{2 \times 2} = \begin{bmatrix} \cos(60^\circ) & -\sin(60^\circ) \\ \sin(60^\circ) & \cos(60^\circ) \end{bmatrix}$$



$$\mathbf{A}_{2 \times 2} = \begin{bmatrix} \cos(60^\circ) & -\sin(60^\circ) \\ \sin(60^\circ) & \cos(60^\circ) \end{bmatrix}, \mathbf{B}_{2 \times 2} = \begin{bmatrix} \cos(45^\circ) & -\sin(45^\circ) \\ \sin(45^\circ) & \cos(45^\circ) \end{bmatrix}$$



$$\mathbf{A}_{2 \times 2} = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}, \mathbf{B}_{2 \times 2} = \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix}$$



$$\mathbf{A}_{2 \times 2} = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}, \mathbf{B}_{2 \times 2} = \begin{bmatrix} 1 & 0 \\ 2 & 1 \end{bmatrix}$$



Q3. Given the column vector $\mathbf{a} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$, compute $\mathbf{a}^T @ \mathbf{a}$ and $\mathbf{a} @ \mathbf{a}^T$.

Q4. Given the column vectors $\mathbf{a} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$ and $\mathbf{b} = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$, compute $\mathbf{a} @ \mathbf{b}^T$ and $\mathbf{b} @ \mathbf{a}^T$.

Q5. Given $\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$ and $\mathbf{b} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$, compute $\mathbf{A} @ \mathbf{b}$ and $\mathbf{b}^T @ \mathbf{A}^T$.

Q6. Given $\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix}$, compute $\mathbf{A} @ \mathbf{A}^T$ and $\mathbf{A}^T @ \mathbf{A}$.

Q7. Use NumPy to compute the matrix multiplication corresponding to each subplot in Figure 9, and verify it using vector inner products.

Q8. Modify Code 2, swap the 1st and 2nd for loops, and compute the matrix multiplication.